Software Engineering

Quick Note Taker App

Pham Trung Hieu
Global IT Engineering

6

Upgrading Quick Note Taker: Menus, Tray, and Multiple Notes

- In this lecture, we will continue upgrading the **Quick Note Taker** app with **four new features**.
- By the end, the app will have a **native menu bar**, a **system tray icon**, and the ability to **store** and manage **multiple notes**.

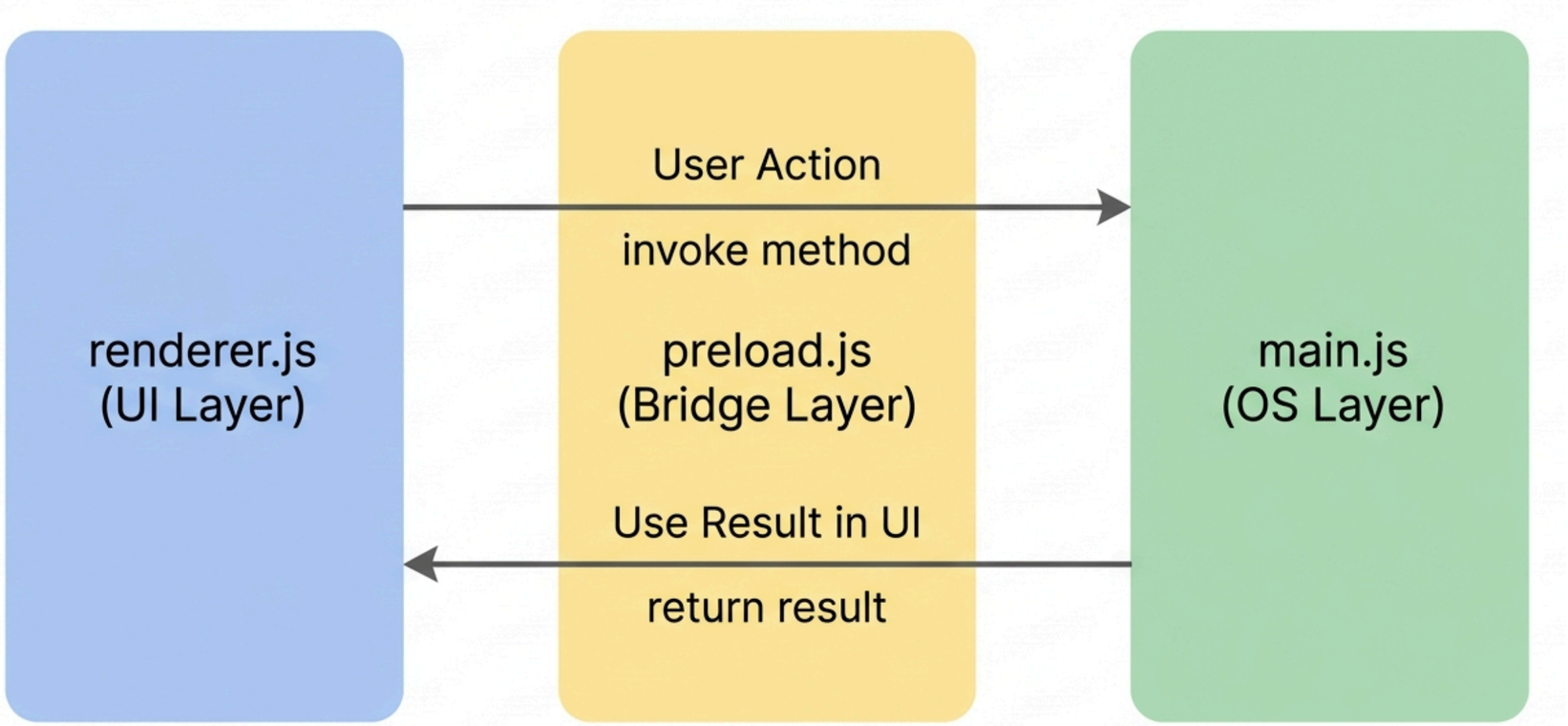
What We Will Build?

- By the end of this lecture, the **Quick Note Taker** app will have:
  - **App Menu**: A native **File menu** in the menu bar with all file actions
  - **System Tray**: A **tray icon** that keeps the app accessible even when the window is closed
  - **Multiple Notes**: Store **many notes** in a **JSON file** instead of a single **.txt** file
  - **Note Sidebar**: A list of all notes, click to open, with delete support
- **Note**: Each of these features builds on what you already know. The **IPC** pattern from **Week 6** is used again in every section. If something feels unfamiliar, refer back to the Week 6 IPC flow diagram.

What the App Can Do So Far?

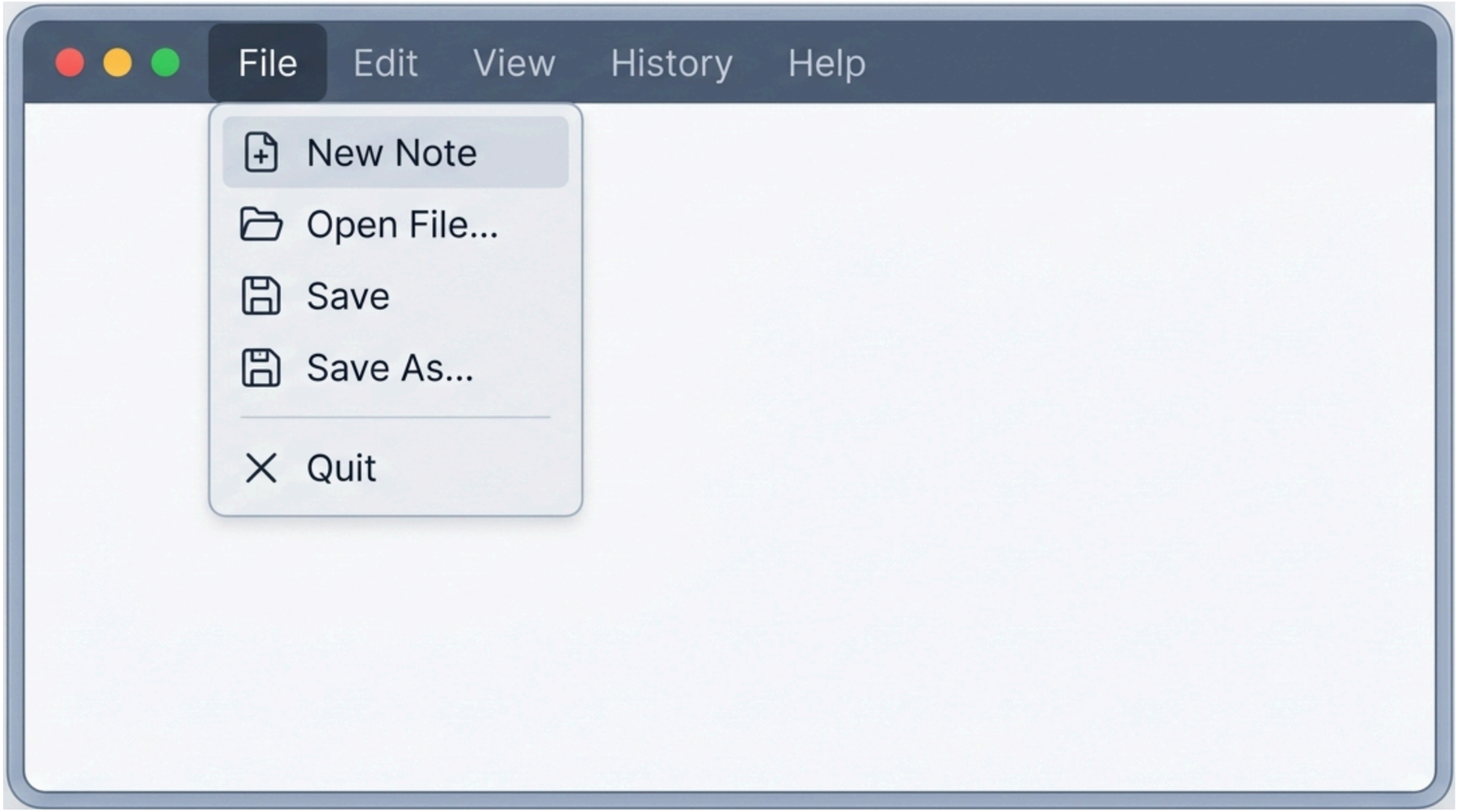
Feature	Where it lives
Auto-save after 5 seconds	renderer.js
Save to a fixed path (quicknote.txt)	main.js + preload.js
Save As → choose file name and location	main.js + preload.js
New Note → with unsaved changes warning	main.js + preload.js
Open File → load any .txt file	main.js + preload.js
Smart Save → save to current file path	main.js + preload.js

IPC Pattern



Note: You will see this pattern repeated four times today, once for each new feature. By the end of this lecture, recognizing this pattern should feel natural and automatic.

SECTION 1: App Menu



App Menu

What is the App Menu?

- The **App Menu** is the **native menu bar** at the **top** of a desktop application window.
  - On **Windows**, it sits inside the app window.
  - On **macOS**, it sits at the top of the screen.
- Right now, **Quick Note Taker** has Electron's default menu, which includes **developer tools** and other options that are not relevant to users.

We will **replace** it with a clean **File menu** that contains:

- 📄 New Note
- 📂 Open File
- 💾 Save
- 📁 Save As
- ——— (separator)
- ❌ Quit

Introducing Menu and MenuItem

- **Electron** provides **two built-in** tools for building menus:
  - **Menu**: Represents the entire menu bar
  - **MenuItem**: Represents a single item inside a menu
- We import them in main.js alongside the existing imports:

```
// BEFORE
const { app, BrowserWindow, ipcMain, dialog } = require('electron');

// AFTER
const { app, BrowserWindow, ipcMain, dialog, Menu } = require('electron');
```

Note: We only need to import Menu → individual menu items are defined as plain JavaScript objects inside a template array. We do not need to import MenuItem separately for this approach.

How Electron Menus Work

Electron menus are built from a template, an array of objects where each object represents one top-level menu (like "**File**" or "**Edit**").

Basic structure:

```
const template = [
    {
        label: 'File',          // the top-level menu name
        submenu: [              // the items inside the menu
            {
                label: 'New Note',
                accelerator: 'CmdOrCtrl+N',   // keyboard shortcut
                click: () => { /* action */ }  // what happens on click
            },
            { type: 'separator' },            // a dividing line
            {
                label: 'Quit',
                accelerator: 'CmdOrCtrl+Q',
                click: () => app.quit()
            }
        ]
    }
];
const menu = Menu.buildFromTemplate(template);
Menu.setApplicationMenu(menu);
```

Step 1: Build the Menu Template in main.js

Add this block in main.js inside the app.whenReady() block, after createWindow():

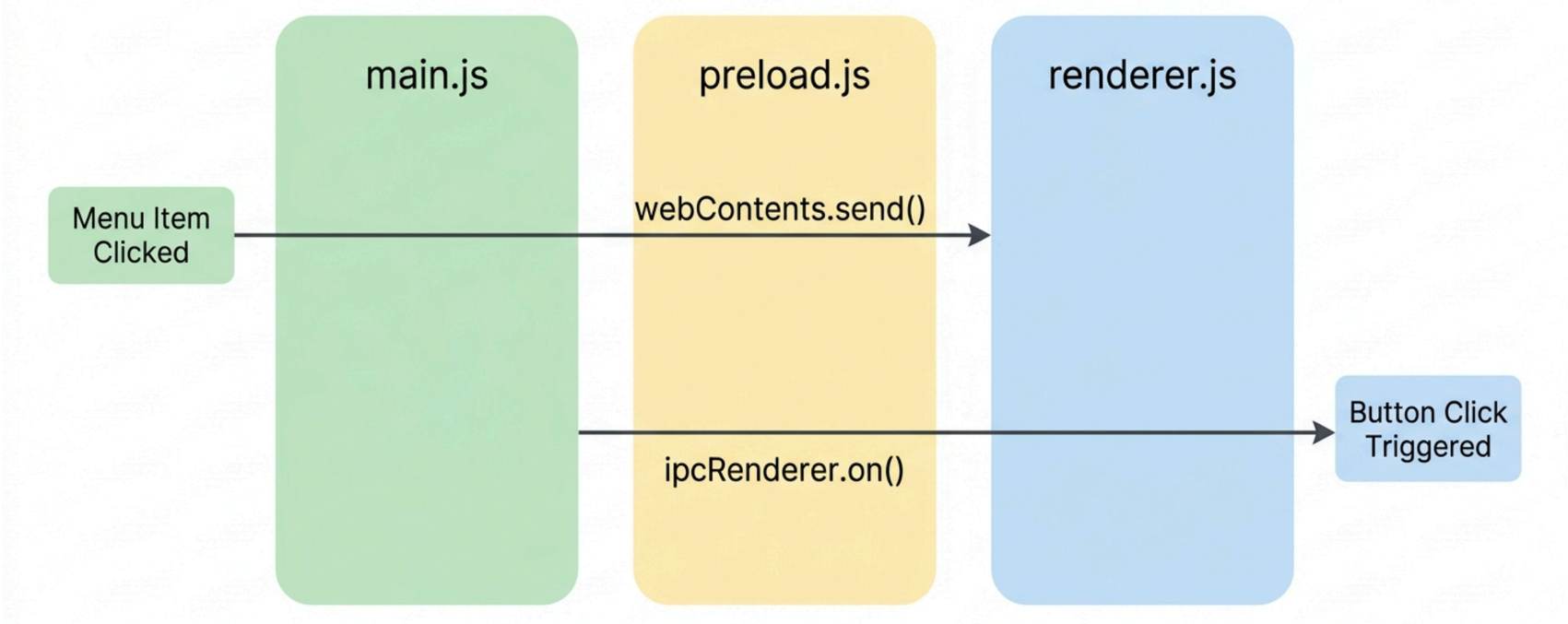
```
// NEW: App Menu
const menuTemplate = [
    {
        label: 'File',
        submenu: [
            {
                label: 'New Note',
                accelerator: 'CmdOrCtrl+N',
                click: () => {
                    BrowserWindow.getFocusedWindow().webContents.send('menu-new-note');
                }
            },
            {
                label: 'Open File',
                accelerator: 'CmdOrCtrl+O',
                click: () => {
                    BrowserWindow.getFocusedWindow().webContents.send('menu-open-file');
                }
            },
```

```
const menuTemplate = [
    {
        label: 'File',
        submenu: [
            {
                label: 'Save',
                accelerator: 'CmdOrCtrl+S',
                click: () => {
                    BrowserWindow.getFocusedWindow().webContents.send('menu-save');
                }
            },
            {
                label: 'Save As',
                accelerator: 'CmdOrCtrl+Shift+S',
                click: () => {
                    BrowserWindow.getFocusedWindow().webContents.send('menu-save-as');
                }
            },
            { type: 'separator' },
            {
                label: 'Quit',
                accelerator: 'CmdOrCtrl+Q',
                click: () => app.quit()
            }
        ]
    }
];
const menu = Menu.buildFromTemplate(menuTemplate);
Menu.setApplicationMenu(menu);
```

Note: Notice that menu item clicks use webContents.send(), this is a **new direction** of communication. Instead of the renderer calling the main process, the main process sends a message to the renderer. This is how menu actions **trigger UI changes**. We will handle these messages in renderer.js next.

Main to Renderer Communication

So far, communication always went from **renderer → main**. Menu clicks introduce the opposite direction:



Note: This is the only time in Electron where the main process starts the conversation. webContents.send() pushes a message to the renderer. The renderer listens with ipcRenderer.on() instead of ipcRenderer.invoke(). **The difference**: **invoke** is for **two-way** communication (**send and wait** for reply), **on** is for **one-way** communication (just listen).

Step 2: Add the Bridge Listeners in preload.js

Add a new **onMenuAction** method to the bridge:

```
contextBridge.exposeInMainWorld('electronAPI', {
    saveNote: (text) => ipcRenderer.invoke('save-note', text),
    loadNote: () => ipcRenderer.invoke('load-note'),
    saveAs: (text) => ipcRenderer.invoke('save-as', text),
    newNote: () => ipcRenderer.invoke('new-note'),
    openFile: () => ipcRenderer.invoke('open-file'),
    smartSave: (text, filePath) => ipcRenderer.invoke('smart-save', text, filePath),
    onMenuAction: (channel, callback) => ipcRenderer.on(channel, callback)  // NEW
});
```

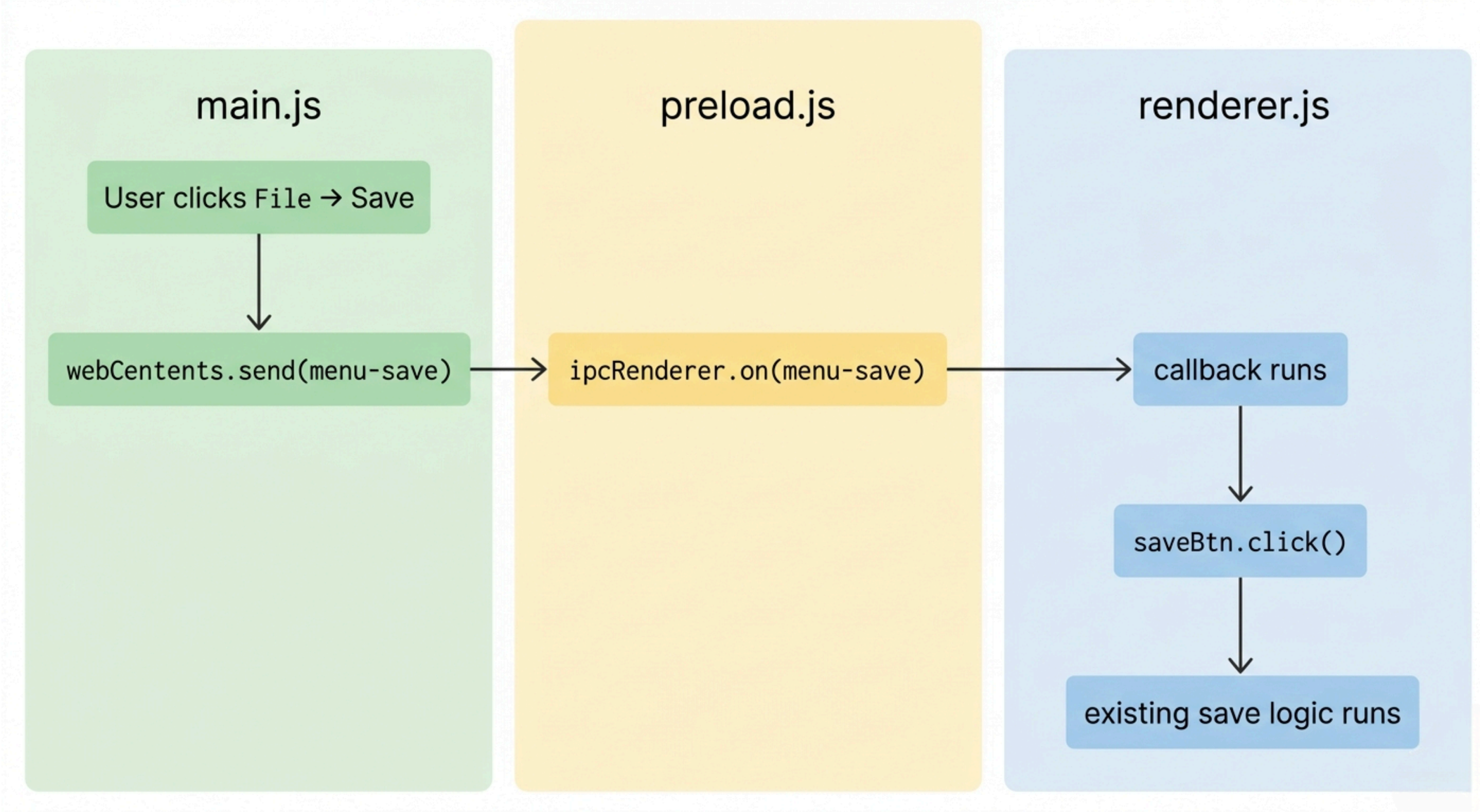
Note: **onMenuAction** takes **two** arguments, the **channel** name ('**menu-save**') and a **callback function** to run when that message arrives. This is a reusable method; we use it once for each menu item in renderer.js instead of writing four separate bridge methods.

Step 3: Listen for Menu Actions in renderer.js

Add these listeners inside DOMContentLoaded, after the existing button listeners:

```
// NEW: Menu action listeners
window.electronAPI.onMenuAction('menu-new-note', () => {
    newNoteBtn.click();    // reuse the existing button logic
});
window.electronAPI.onMenuAction('menu-open-file', () => {
    openFileBtn.click();   // reuse the existing button logic
});
window.electronAPI.onMenuAction('menu-save', () => {
    saveBtn.click();       // reuse the existing button logic
});
window.electronAPI.onMenuAction('menu-save-as', () => {
    saveAsBtn.click();     // reuse the existing button logic
});
```

App Menu: Full Flow Diagram



Practice Task 1

Task: Add the **File menu** to your **Quick Note Taker** app.

Follow these steps:

1. Add **Menu** to the import line in main.js
2. Build the menu template and set it as the application menu in main.js
3. Add the **onMenuAction** bridge method in preload.js
4. Add the four menu action listeners in renderer.js
5. Run the app and test each menu item and its keyboard shortcut

SECTION 2 — System Tray

What is the System Tray?

The **System Tray** is the area at the **bottom-right** of the screen on **Windows**, or the **top-right** on **macOS**. It shows small icons for apps that run in the background.

We will add a **tray icon** to **Quick Note Taker** so that:

- The **app icon** always appears in the **system tray**
- **Right-clicking** the tray icon shows a context menu
- The context **menu** has **options** to show the **window** and **quit** the app
- **Closing** the **window** hides it instead of quitting the app

Note: Apps like **Slack**, **Dropbox**, and **Spotify** use the system tray to stay accessible without taking up space in the taskbar. After this section, **Quick Note Taker** will behave the same way.

Introducing the Tray Module

Electron provides a built-in Tray class for creating system tray icons.

Add Tray to the import line in main.js:

```
// BEFORE
const { app, BrowserWindow, ipcMain, dialog, Menu } = require('electron');

// AFTER
const { app, BrowserWindow, ipcMain, dialog, Menu, Tray } = require('electron');
```

The tray also needs an icon image. We will create a simple icon file and place it in the project folder. For now, we will use a placeholder path:

```
const tray = new Tray('/path/to/icon.png');
```

Note: The **tray icon** must be a **real image file.** Electron cannot generate one automatically.

For **Windows**, **.ico** files work best. For **macOS**, **.png** files at **16x16** or **32x32 pixels** work well.

For this lecture, we will use a small .png file placed in the project root folder.

Step 1: Create the Tray in main.js

Add this code inside **app.whenReady()**, after the menu setup:

Note: **setToolTip** sets the text that appears when the user hovers over the tray icon. **setContextMenu** attaches the right-click menu to the tray icon.

BrowserWindow.getAllWindows()[0] gets the first (and only) window in our app.

```
// NEW: System Tray
let tray = null;
app.whenReady().then(() => {
    createWindow();
    // ... menu setup code ...
    // Create tray icon
    tray = new Tray(path.join(__dirname, 'tray-icon.png'));
    // Tray context menu
    const trayMenu = Menu.buildFromTemplate([
        {
            label: 'Show App',
            click: () => {
                BrowserWindow.getAllWindows()[0].show();
            }
        },
        {
            label: 'Quit',
            click: () => app.quit()
        }
    ]);
    tray.setToolTip('Quick Note Taker');
    tray.setContextMenu(trayMenu);
});
```

Step 2: Hide the Window Instead of Closing It

Right now, clicking the window's close button quits the app. We want it to hide the window instead, keeping the app running in the tray.

Add this code inside **createWindow()**, after **win.loadFile**('index.html'):

```
// NEW: Hide window instead of closing
win.on('close', (event) => {
    event.preventDefault();   // stop the window from actually closing
    win.hide();               // hide it instead
});
```

Note: **event.preventDefault()** cancels the default close behavior. Without this line, the window would close and the app would quit. With it, the window simply disappears from view but the app keeps running. The user can bring it back by clicking "**Show App**" in the **tray menu**.

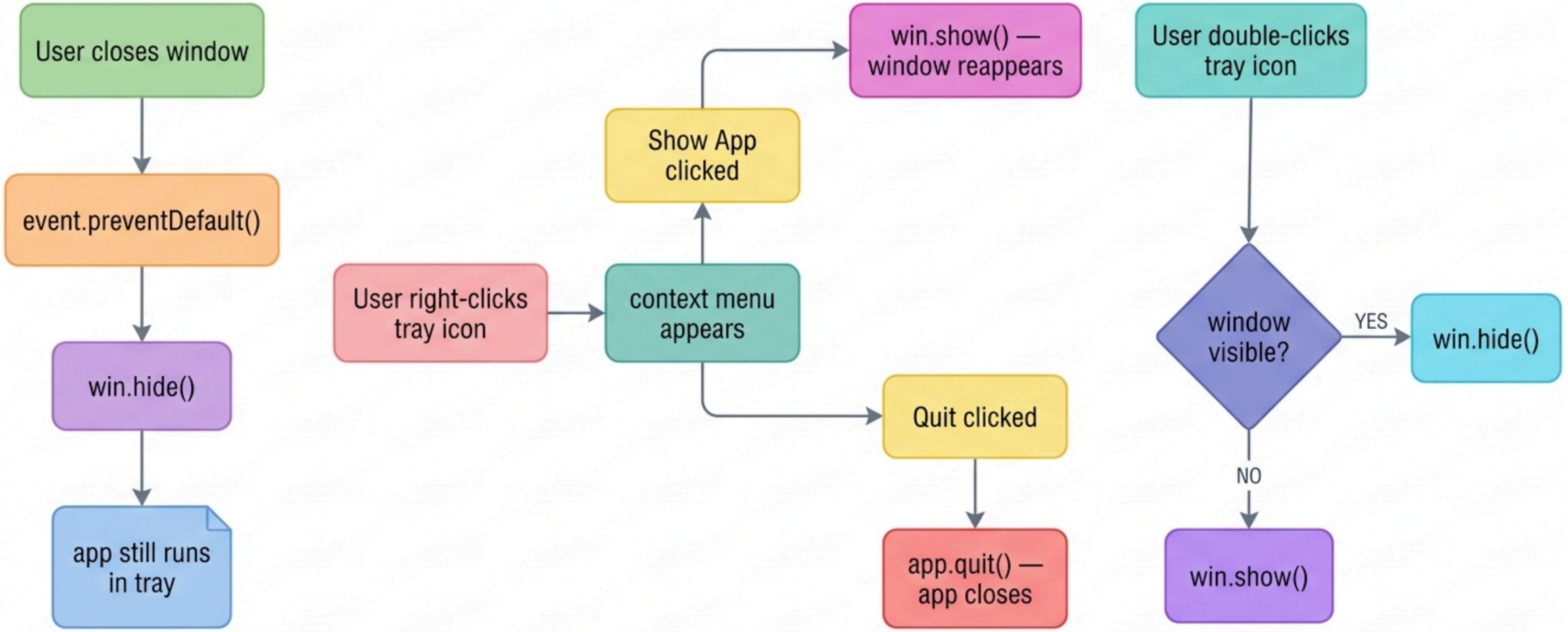
Step 3: Double-Click Tray Icon to Show Window

Add a **double-click** listener to the tray so users can quickly show the window:

```
// NEW: Double-click tray icon to show window
tray.on('double-click', () => {
    const win = BrowserWindow.getAllWindows()[0];
    if (win.isVisible()) {
        win.hide();
    } else {
        win.show();
    }
});
```

Note: This adds a toggle behavior; **double-clicking** the tray icon shows the window if it is hidden, or hides it if it is visible. This is a common pattern in tray applications and feels natural to users.

System Tray: Full Flow Diagram



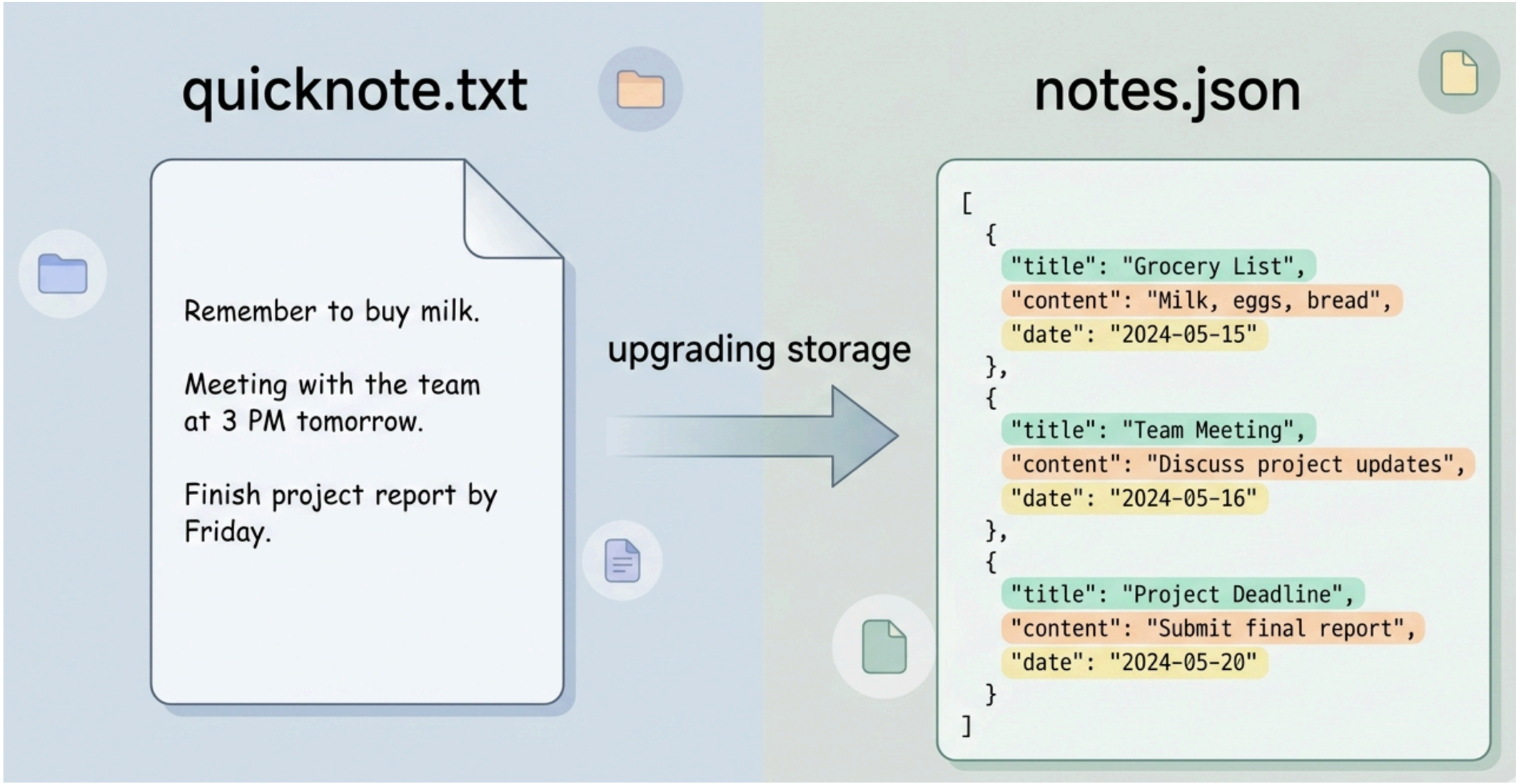
Practice Task 2

Task: Add the system tray to your **Quick Note Taker** app.

Follow these steps:

1. Design a **tray icon image** and save it as **tray-icon.png** in your **project folder**
2. Add Tray to the import line in main.js
3. Create the tray with a context menu inside **app.whenReady()**
4. Add the close **event handler** to hide the window instead of quitting
5. Add the **double-click** handler to toggle the window
6. Run the app, close the window, and confirm the tray icon remains

SECTION 3 — Storing Multiple Notes with JSON

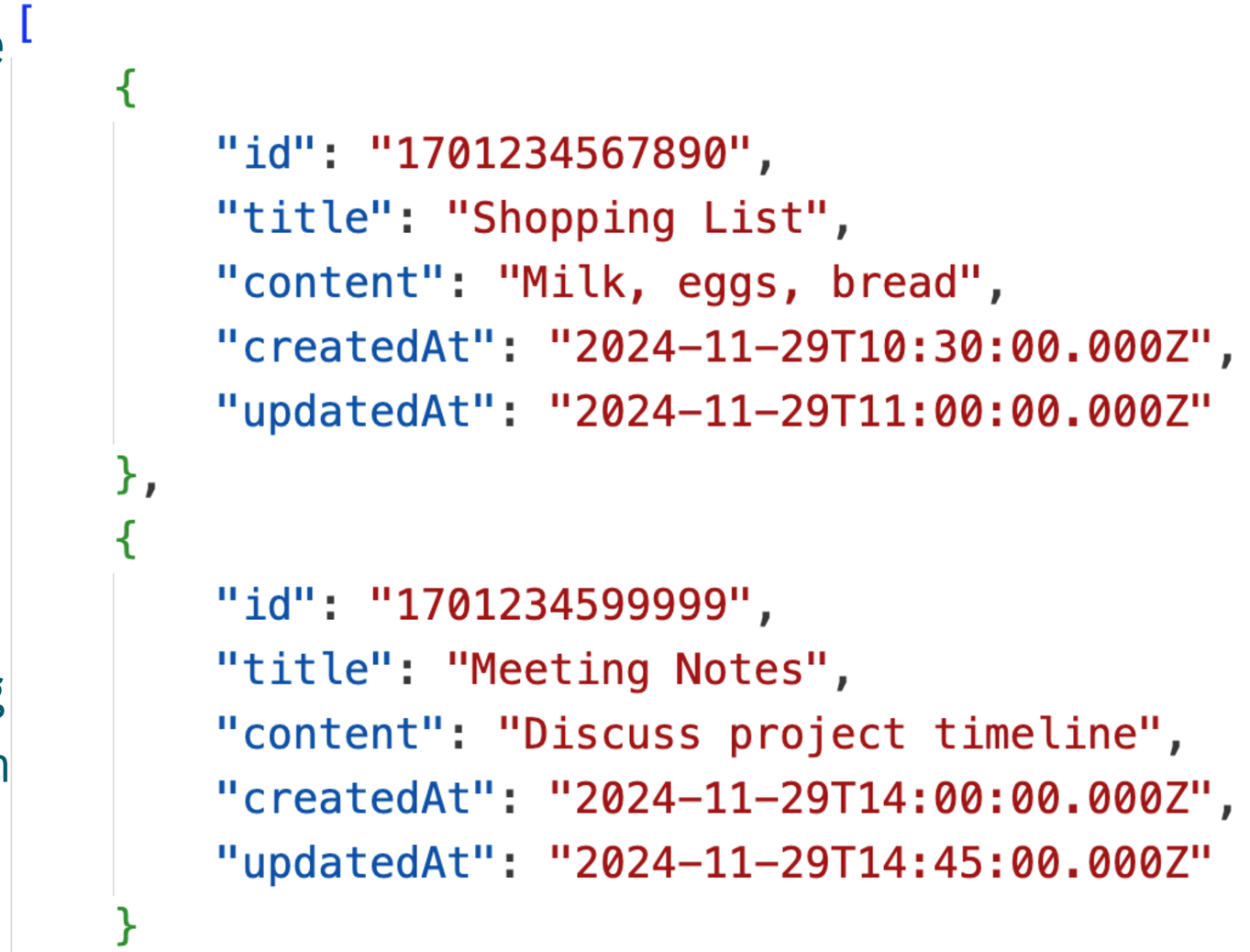


Why JSON Instead of a Single .txt File?

The current app stores only one note in one .txt file. To support multiple notes, we need a better storage format.

We will switch to a **JSON** file that stores all notes together:

Note: JSON stands for **JavaScript Object Notation**. It is a standard format for storing structured data. Each note is an object with **five fields**: a unique **id**, a **title**, the **note content**, a **createdAt timestamp**, and an **updatedAt timestamp**. The whole file is an **array [ ]** of note objects.



Understanding the Note Structure

Each note has **five fields**:

Field	Type	Description
id	string	A unique identifier, we use the current timestamp in milliseconds
title	string	The note title shown in the sidebar
content	string	The full text of the note
createAt	string	When the note was first created (ISO date format)
updateAt	string	When the note was last saved

Step 1: Define the Notes File Path in main.js

Add this line near the top of main.js, after the existing require statements:

```
// NEW: Path for the notes JSON file
const notesFilePath = path.join(app.getPath('userData'), 'notes.json');
```

Note: We use **app.getPath**('userData') instead of **app.getPath**('documents'). userData is a special folder that Electron **reserves** for each app's private data. On **Windows** it is inside **AppData/Roaming**. On **macOS** it is inside **Library/Application Support**. Users do not normally browse this folder, which makes it a good place for app data files.

Step 2: Add Helper Functions in main.js

Add these two helper functions in main.js, before the **IPC handlers**:

```
// NEW: Helper — read all notes from the JSON file
function readNotes() {
    if (!fs.existsSync(notesFilePath)) {
        return [];    // return empty array if file does not exist yet
    }
    const raw = fs.readFileSync(notesFilePath, 'utf-8');
    return JSON.parse(raw);
}

// NEW: Helper — write all notes to the JSON file
function writeNotes(notes) {
    fs.writeFileSync(notesFilePath, JSON.stringify(notes, null, 2), 'utf-8');
}
```

Note: **JSON.parse()** converts a **JSON** string from the file into a **JavaScript** array we can work with. **JSON.stringify**(notes, null, 2) **converts** the **array** back into a **formatted JSON** string → the 2 adds indentation to make the file **human-readable**. These two helper functions keep our **IPC** handlers **clean** and simple.

Step 3: Add IPC Handlers for Notes in main.js

Add these **four handlers** after the existing handlers:

```
// NEW: Get all notes
ipcMain.handle('get-notes', async () => {
    return readNotes();
});

// NEW: Delete a note
ipcMain.handle('delete-note', async (event, id) => {
    const notes = readNotes();
    const filtered = notes.filter(n => n.id !== id);
    writeNotes(filtered);
    return { success: true };
});
```

Step 3: Add IPC Handlers for Notes in main.js

```
// NEW: Save a note (create or update)
ipcMain.handle('save-note-json', async (event, note) => {
    const notes = readNotes();
    const index = notes.findIndex(n => n.id === note.id);
    const now = new Date().toISOString();

    if (index === -1) {
        // Note does not exist yet – create it
        notes.push({ ...note, createdAt: now, updatedAt: now });
    } else {
        // Note already exists – update it
        notes[index] = { ...notes[index], ...note, updatedAt: now };
    }

    writeNotes(notes);
    return { success: true };
});
```

Step 3: Add IPC Handlers for Notes in main.js

Note: **findIndex** searches the **array** for a note with a matching id. If it returns **-1**, the note **does not exist** yet and we create it. If it returns a number, the note exists and we update it. **filter** removes the note with the matching **id** for deletion. The spread operator ... **copies** all existing fields before **overwriting** with new values.

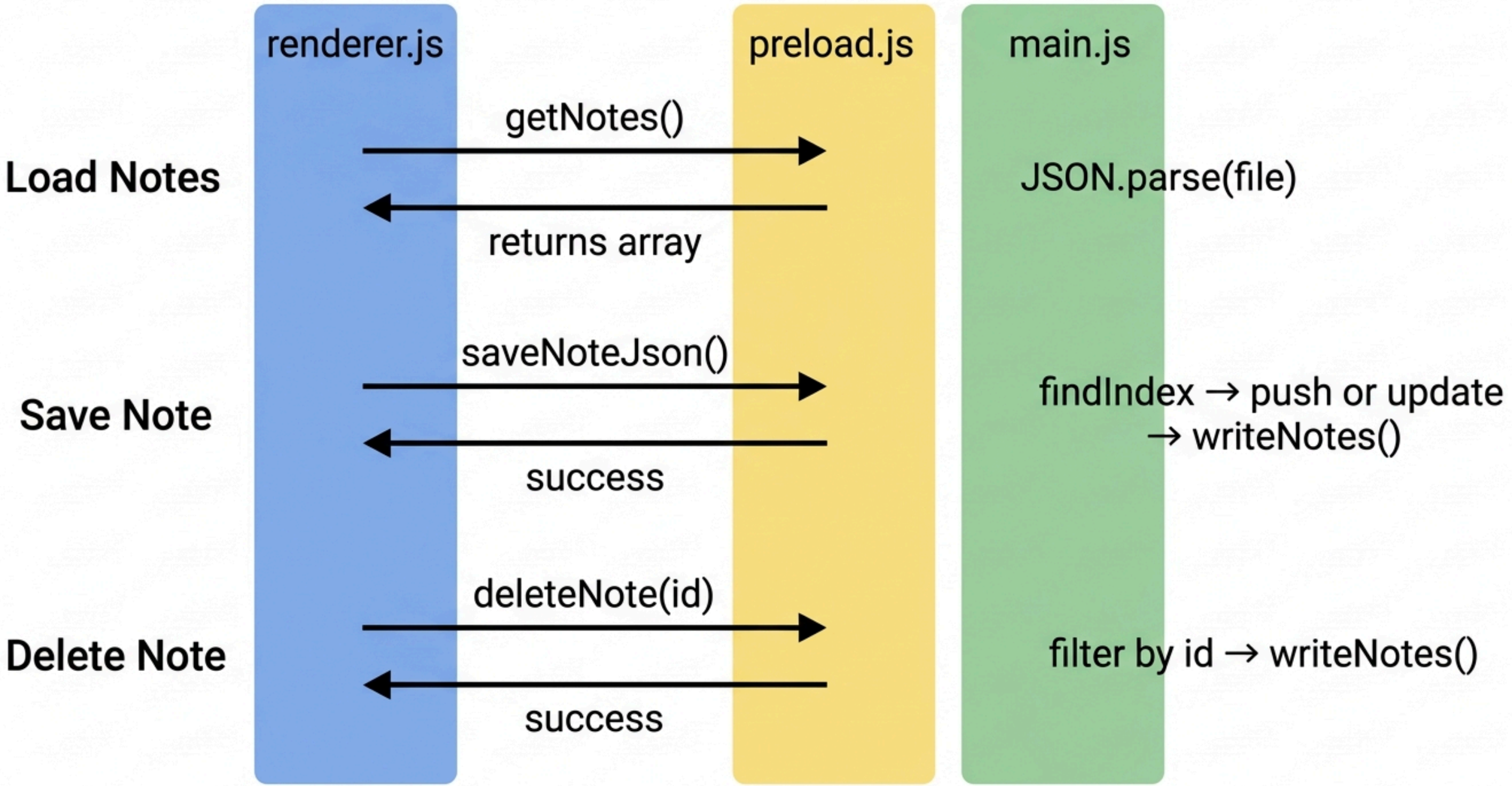
Step 4: Update preload.js with New Bridge Methods

Add the **three** new **methods** to the bridge:

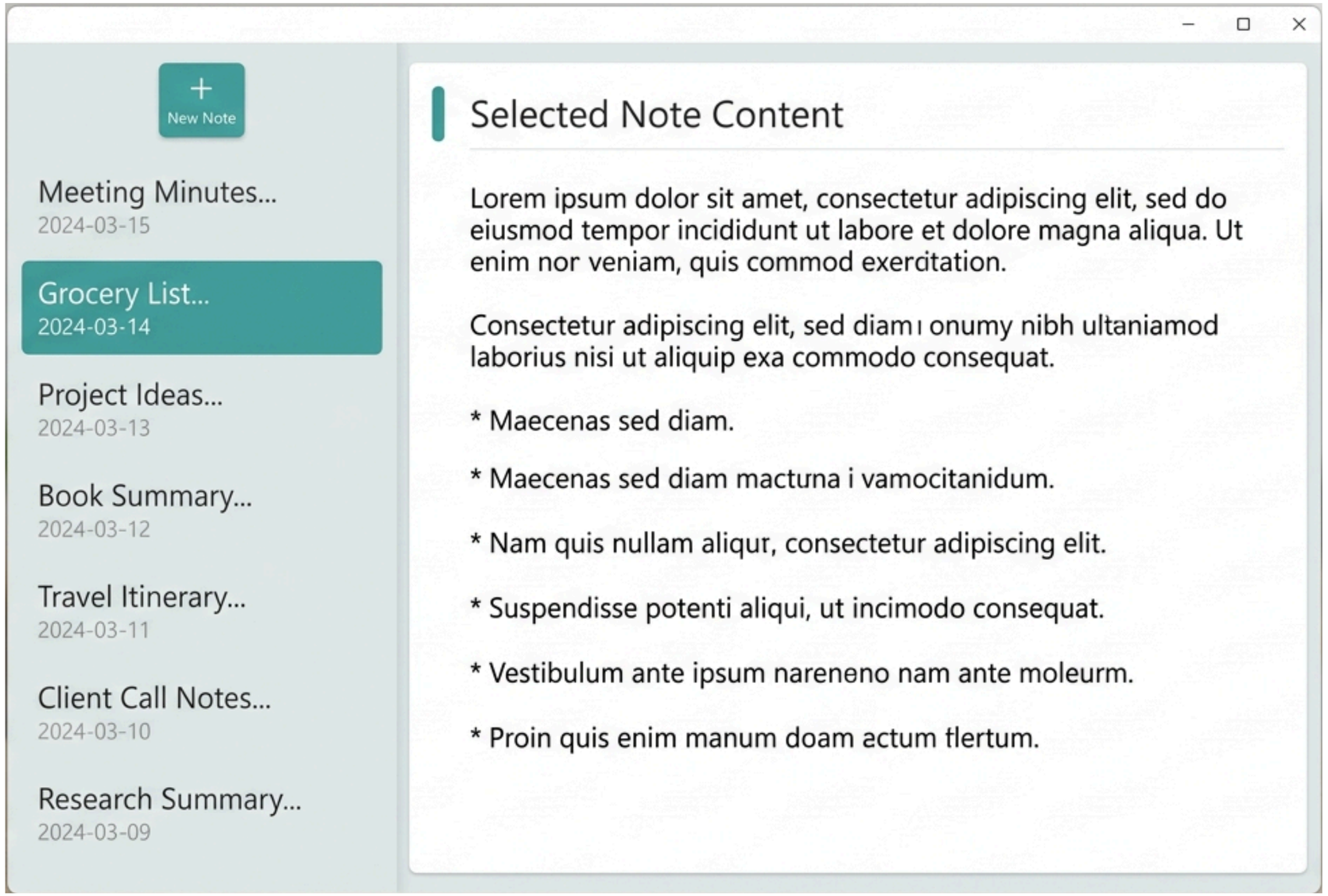
```
contextBridge.exposeInMainWorld('electronAPI', {
    // existing methods
    saveNote: (text) => ipcRenderer.invoke('save-note', text),
    loadNote: () => ipcRenderer.invoke('load-note'),
    saveAs: (text) => ipcRenderer.invoke('save-as', text),
    newNote: () => ipcRenderer.invoke('new-note'),
    openFile: () => ipcRenderer.invoke('open-file'),
    smartSave: (text, filePath) => ipcRenderer.invoke('smart-save', text, filePath),
    onMenuAction: (channel, callback) => ipcRenderer.on(channel, callback),

    // NEW: JSON notes methods
    getNotes: () => ipcRenderer.invoke('get-notes'),
    saveNoteJson: (note) => ipcRenderer.invoke('save-note-json', note),
    deleteNote: (id) => ipcRenderer.invoke('delete-note', id)
});
```

JSON Notes: Full Flow Diagram



SECTION 5 — Note List Sidebar



What the Sidebar Will Look Like

The **sidebar** will appear on the **left side** of the app. It will show a **list** of all **saved notes**.

Each note in the list will show:

- The note **title**
- The last **modified date**
- A **delete** button

At the top of the **sidebar,** there will be a **New Note button**.

Clicking a **note** in the list will **load** it into the **editor** on the right.

Note: This transforms **Quick Note Taker** from a single-note app into a real multi-note manager, similar to how the **Windows Sticky Notes** app or **Apple Notes** works.

Step 1: Update index.html for the Sidebar Layout

Replace the existing <body> content with this new layout:

```
<body>
    <div id="app">
        <!-- Sidebar -->
        <div id="sidebar">
            <button id="new-note">📄 New Note</button>
            <div id="note-list"></div>
        </div>

        <!-- Editor -->
        <div id="editor">
            <input type="text" id="note-title" placeholder="Note title...">
            <textarea id="note" placeholder="Start typing your note..."></textarea>
            <div id="toolbar">
                <button id="save">💾 Save</button>
                <button id="save-as">📁 Save As</button>
                <button id="open-file">📂 Open File</button>
            </div>
            <p id="save_status"></p>
        </div>
    </div>
    <script src="renderer.js"></script>
</body>
```

Note: We added a title input field (#note-title) above the **textarea**. This lets users give each note a meaningful name. The layout is now split into **two columns**, **#sidebar** on the **left** and **#editor** on the **right**. We will use **CSS** to arrange them side by side.

Step 2: Add CSS for the Two-Column Layout in index.html

```
<style>
    * { box-sizing: border-box; margin: 0; padding: 0; }

    body { font-family: system-ui; background: #f4f4f4; height: 100vh; }

    #app {
    display: flex;
    height: 100vh;
    }

    #sidebar {
    width: 220px;
    background: #2b2b2b;
    color: white;
    display: flex;
    flex-direction: column;
    padding: 10px;
    gap: 8px;
    }
```

Replace the existing <style> block with this updated version:

```css
#note-list {
flex: 1;
overflow-y: auto;
display: flex;
flex-direction: column;
gap: 6px;
}

#editor {
flex: 1;
display: flex;
flex-direction: column;
padding: 15px;
gap: 10px;
}
```

```css
#note-title {
font-size: 18px;
padding: 8px;
border: 1px solid #ccc;
border-radius: 6px;
}

textarea {
flex: 1;
font-size: 16px;
padding: 12px;
resize: none;
border: 1px solid #ccc;
border-radius: 6px;
}

#toolbar { display: flex; gap: 8px; }
```

```css
button {
padding: 8px 12px;
font-size: 14px;
background: #316357;
color: white;
border: none;
cursor: pointer;
border-radius: 8px;
}

button:hover { background: #3d7a6b; }

#new-note { width: 100%; margin-bottom: 6px; }

.note-item {
background: #3a3a3a;
border-radius: 6px;
padding: 8px;
cursor: pointer;
}
```

```
.note-item:hover { background: #4a4a4a; }
.note-item.active { background: #316357; }
.note-item-title {
font-size: 13px;
font-weight: bold;
white-space: nowrap;
overflow: hidden;
text-overflow: ellipsis;
}
.note-item-date { font-size: 11px; color: #aaa; margin-top: 3px; }
.note-item-delete {
background: transparent;
color: #ff6b6b;
border: none;
font-size: 12px;
float: right;
padding: 0;
cursor: pointer;
}
#save_status { font-size: 13px; color: gray; }
```

Note: display: flex on #app places the sidebar and editor side by side. flex: 1 on #editor makes it fill all remaining space after the sidebar. text-overflow: ellipsis truncates long note titles with … so they do not overflow the **sidebar**.

Step 3: Rewrite renderer.js - State Variables

Replace the top of the DOMContentLoaded block with these updated state variables:

Note: We now track an array of notes and the currentNoteId instead of a file path. The lastSavedContent variable replaces **lastSavedText**, same idea, new name to match the new data model.

```
window.addEventListener('DOMContentLoaded', async () => {
    const textarea = document.getElementById('note');
    const titleInput = document.getElementById('note-title');
    const saveBtn = document.getElementById('save');
    const saveAsBtn = document.getElementById('save-as');
    const openFileBtn = document.getElementById('open-file');
    const newNoteBtn = document.getElementById('new-note');
    const noteList = document.getElementById('note-list');
    const statusEl = document.getElementById('save_status');

    // State
    let notes = [];                    // all notes loaded from JSON
    let currentNoteId = null;          // id of the note being edited
    let lastSavedContent = '';         // tracks unsaved changes
    let debounceTimer = null;
```

Step 4: Add the renderNoteList Function in renderer.js

Add this function inside DOMContentLoaded:

```
// NEW: Render the note list in the sidebar
function renderNoteList() {
  noteList.innerHTML = '';    // clear existing list
  notes.forEach(note => {
    const item = document.createElement('div');
    item.className = 'note-item' + (note.id === currentNoteId ? ' active' : '');
    item.innerHTML = `
          <button class="note-item-delete" data-id="${note.id}">x</button>
          <div class="note-item-title">${note.title || 'Untitled'}</div>
          <div class="note-item-date">${new Date(note.updatedAt).toLocaleDateString()}</div>
      `;
    // Click note to open it
    item.addEventListener('click', async (e) => {
      if (e.target.classList.contains('note-item-delete')) return;
      await switchNote(note.id);
    });
    // Delete button
    item.querySelector('.note-item-delete').addEventListener('click', async (e) => {
      e.stopPropagation();
      await deleteNote(note.id);
    });
    noteList.appendChild(item);
  });
}
```

Note: noteList.innerHTML = '' clears the list before redrawing it; this is the simplest way to refresh the **sidebar**. The active class **highlights** the currently **selected note**. e.stopPropagation() on the delete button prevents the click from also triggering the note open logic.

Step 5: Add the switchNote Function in renderer.js

```
// NEW: Switch to a different note (with unsaved changes warning)
async function switchNote(id) {
  // Check for unsaved changes first
  if (textarea.value !== lastSavedContent) {
    const result = await window.electronAPI.newNote();
    if (!result.confirmed) return;    // user cancelled — stay on current note
  }

  // Load the selected note
  const note = notes.find(n => n.id === id);
  if (!note) return;

  currentNoteId = note.id;
  titleInput.value = note.title || '';
  textarea.value = note.content || '';
  lastSavedContent = note.content || '';
  statusEl.textContent = '';

  renderNoteList();    // refresh sidebar to show active state
}
```

Add this function after renderNoteList:

Note: We reuse window.electronAPI.newNote() for the **unsaved** changes warning; the same dialog from Week 6. This is good design: one dialog, used in multiple places. notes.find(n => n.id === id) searches the array for the note with the matching id and returns it.

Step 6: Add the saveCurrentNote Function in renderer.js

```
// NEW: Save the currently open note to JSON
async function saveCurrentNote() {
    if (!currentNoteId) return;
    const note = {
        id: currentNoteId,
        title: titleInput.value || 'Untitled',
        content: textarea.value
    };
    await window.electronAPI.saveNoteJson(note);
    lastSavedContent = textarea.value;
    // Update the note in the local array too
    const index = notes.findIndex(n => n.id === currentNoteId);
    if (index !== -1) {
        notes[index] = { ...notes[index], ...note, updatedAt: new Date().toISOString() };
    }
    renderNoteList();
    statusEl.textContent = `Saved at ${new Date().toLocaleTimeString()}`;
}
```

Add this function after switchNote:

Note: After saving to the file, we also update the notes array in memory. This keeps the sidebar in sync without needing to reload the entire file from disk. { ...notes[index], ...note } merges the existing note data with the updated fields.

Step 7: Add the deleteNote Function in renderer.js

Add this function after saveCurrentNote:

```
// NEW: Delete a note
async function deleteNote(id) {
    const result = await window.electronAPI.newNote();  // reuse warning dialog
    if (!result.confirmed) return;
    await window.electronAPI.deleteNote(id);
    notes = notes.filter(n => n.id !== id);
    // If we deleted the current note, clear the editor
    if (currentNoteId === id) {
        currentNoteId = null;
        titleInput.value = '';
        textarea.value = '';
        lastSavedContent = '';
        statusEl.textContent = 'Note deleted.';
    }
    renderNoteList();
}
```

Note: We reuse the newNote() warning dialog again here as a "are you sure?" confirmation before deleting. After deletion, we filter the note out of the local notes array so the sidebar updates immediately without reading the file again.

Step 8: Update the New Note Button in renderer.js

Replace the existing New Note button listener with this updated version:

Note: Date.now().toString() generates a unique id based on the current time in **milliseconds**. notes.unshift(newNote) adds the new note to the beginning of the array so it appears at the top of the sidebar. titleInput.focus() moves the cursor to the title field so the user can start typing the title immediately.

```
// UPDATED: New Note button — creates a new note in JSON storage
newNoteBtn.addEventListener('click', async () => {
    if (textarea.value !== lastSavedContent) {
        const result = await window.electronAPI.newNote();
        if (!result.confirmed) return;
    }
    // Create a new note object
    const newNote = {
        id: Date.now().toString(),
        title: 'Untitled',
        content: '',
        createdAt: new Date().toISOString(),
        updatedAt: new Date().toISOString()
    };
    await window.electronAPI.saveNoteJson(newNote);
    notes.unshift(newNote);        // add to the top of the list
    currentNoteId = newNote.id;
    titleInput.value = '';
    textarea.value = '';
    lastSavedContent = '';
    renderNoteList();
    titleInput.focus();            // move cursor to title field
    statusEl.textContent = 'New note created.';
});
```

Step 9: Update the Save Button in renderer.js

Replace the existing save button listener:

```
// UPDATED: Save button
saveBtn.addEventListener('click', async () => {
    await saveCurrentNote();
});
```

Step 10: Add Auto-Save for the New System in renderer.js

Replace the existing auto-save logic:

```
// UPDATED: Auto-save with debounce
textarea.addEventListener('input', () => {
    statusEl.textContent = 'Unsaved changes...';
    clearTimeout(debounceTimer);
    debounceTimer = setTimeout(saveCurrentNote, 5000);
});

// Also auto-save when title changes
titleInput.addEventListener('input', () => {
    clearTimeout(debounceTimer);
    debounceTimer = setTimeout(saveCurrentNote, 5000);
});
```

Step 11: Load Notes on Startup in renderer.js

Replace the existing startup loading code with this:

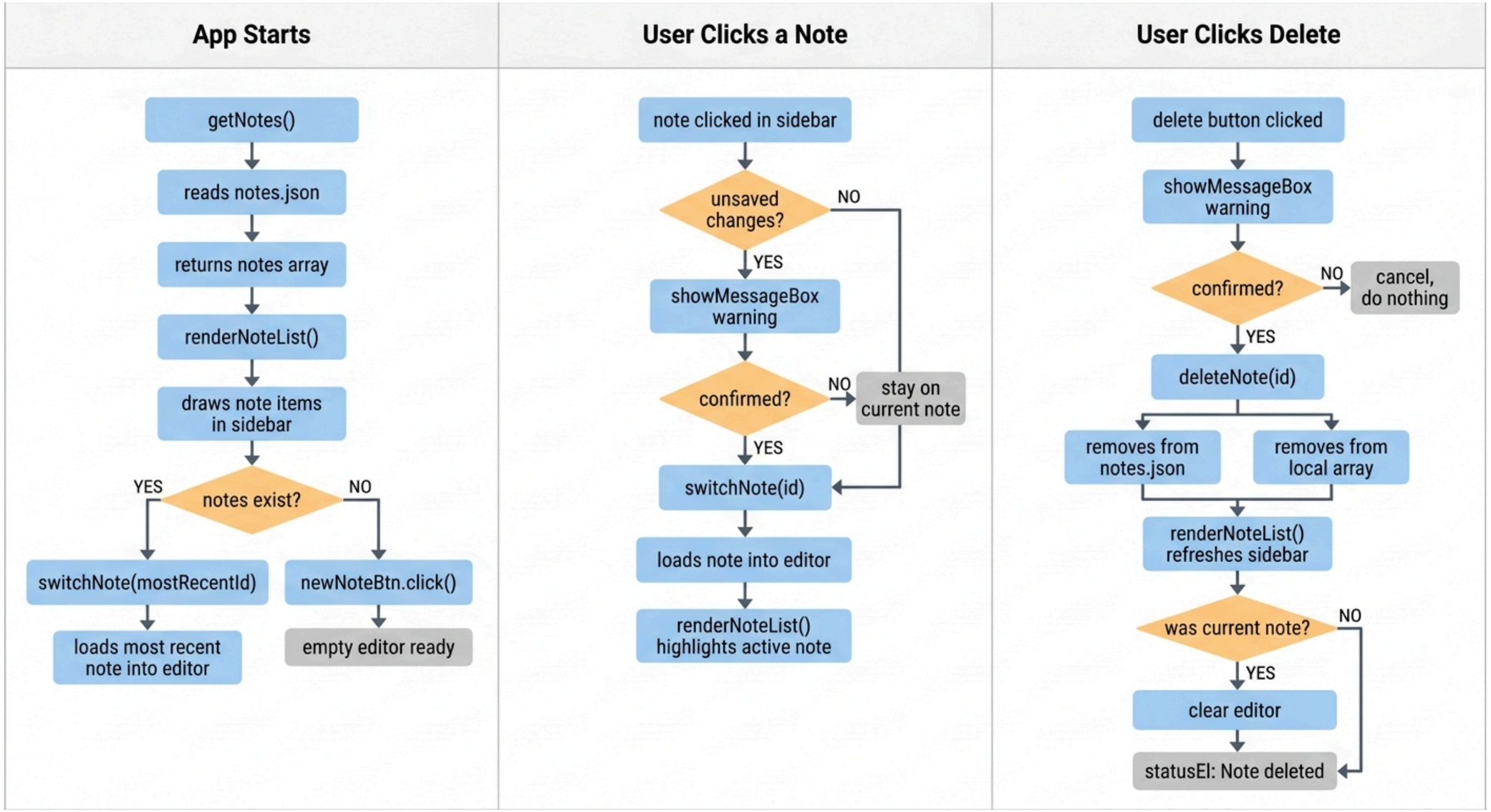
```
// UPDATED: Load all notes on startup
notes = await window.electronAPI.getNotes();

if (notes.length > 0) {
    // Open the most recently updated note
    const mostRecent = notes.reduce((a, b) =>
        new Date(a.updatedAt) > new Date(b.updatedAt) ? a : b
    );
    await switchNote(mostRecent.id);
} else {
    // No notes yet — trigger New Note automatically
    newNoteBtn.click();
}

renderNoteList();
```

Note: reduce compares all notes and returns the one with the most recent updatedAt date. If there are no notes at all, we automatically create a new one so the editor is never empty when the app starts.

Sidebar: Full Flow Diagram



Practice Task 3

Task: Add the **JSON** storage and note **sidebar** to your app.

Follow these steps:

1. Add notesFilePath, readNotes(), and writeNotes() in main.js
2. Add the three JSON IPC handlers in main.js
3. Add the three new bridge methods in preload.js
4. Update index.html with the new layout and CSS
5. Rewrite renderer.js with all the new functions
6. Run the app, create **three notes**, switch between them, and delete one

What We Learned

Feature	Where it lives
App Menu	Menu, Menu.buildFromTemplate(), webContents.send(), ipcRenderer.on()
System Tray	Tray, win.hide(), event.preventDefault(), toggle visibility
JSON Storage	JSON.parse(), JSON.stringify(), findIndex(), filter()
Note Sidebar	Dynamic DOM with createElement, reduce(), state management

Key Concepts to Remember

- The App Menu lives in main.js; menu clicks use webContents.send() to talk to the renderer.
- ipcRenderer.on() listens for messages from main; unlike invoke, it does not send a reply.
- The Tray requires a real image file; use app.getPath('userData') for app data files.
- event.preventDefault() on the close event stops the window from quitting
- JSON is the standard way to store structured data. JSON.parse() reads it, JSON.stringify() writes it.
- Always update both the file and the in-memory array after saving or deleting. This keeps the UI in sync.